Question 5: SQL Query Evaluation

Question 5: SQL Query Evaluation

Assumptions & Gaps Identified

Gaps in the brief that required assumptions:

Gap	Assumption Made	Rationale
Brief says "daily consumption" but readings are 15-minute	Assumed aggregation from READINGS_15MIN to daily totals	As Lead SA: Raw data is 15-min granular, must aggregate to daily
Brief doesn't define "wrong tariff price"	Assumed "wrong" means reading was billed at different price than what should have applied based on effective dates	Late readings may be billed at current price instead of historical price
Brief doesn't specify time period for queries	Assumed queries run against last 90 days (partition-aware)	Performance consideration - full table scan would be too slow
Brief doesn't clarify if "historical tariff edit" means retroactive changes	Assumed Tariff Manager can update effective_from to past date, affecting already-billed readings	This is the "retroactive price change" scenario
Brief doesn't specify if standard deviation should be per day or overall	Assumed overall standard deviation per meter across all time	More useful for anomaly detection than daily variance

Query 1: Average Daily Consumption Per Meter

Reasoning

- Must aggregate 15-minute readings (96/day) to daily totals
- Calculate average across all days
- Only include VALID or BILLED readings (exclude ERROR, REJECTED)
- Group by meter to get per-meter averages

SQL Query

```sql
SELECT
    meter_id,
    ROUND(AVG(daily_kwh), 2) AS avg_daily_kwh,
    COUNT(DISTINCT usage_date) AS days_counted
FROM (
    -- Subquery: Aggregate 15-min readings to daily totals
    SELECT
        meter_id,
        DATE(reading_timestamp) AS usage_date,
        SUM(kwh_value) AS daily_kwh
    FROM READINGS_15MIN
    WHERE status IN ('VALID', 'BILLED')
      AND reading_timestamp >= CURRENT_DATE - INTERVAL 90 DAY  -- Performance: Last 90 days only
    GROUP BY meter_id, DATE(reading_timestamp)
) daily_totals
GROUP BY meter_id
ORDER BY avg_daily_kwh DESC;

```

Explanation

1. **Inner query:** Converts 15-minute readings to daily totals (`SUM(kwh_value)` per day per meter)
2. **Outer query:** Calculates average of daily totals per meter
3. **Filter on status:** Only valid/billed readings (excludes errors and rejections)
4. **Performance:** 90-day window ensures partition pruning
5. **days_counted:** Shows how many days contributed to average (useful for new meters)

Sample Output

```
meter_id  | avg_daily_kwh | days_counted
----------|---------------|-------------
M12345    | 45.67         | 89
M67890    | 123.45        | 90
M11111    | 12.34         | 45

```

Query 2: Find Readings Billed with Wrong Tariff Price (Due to Late Data)

Reasoning

Problem: Late reading arrives after billing run. System may bill it at:

- Current price (at time of billing)
- Instead of historical price (at time of actual usage)

Solution: Compare `billed_at_price` (what we charged) vs correct historical price (what we should have charged)

SQL Query

```sql
SELECT
```

```
    r.reading_id,
    r.meter_id,
    r.reading_timestamp AS usage_timestamp,
    r.received_at,
    DATEDIFF(r.received_at, r.reading_timestamp) AS days_late,
    r.billed_at_price AS price_charged,
    ph.price_per_kwh AS price_should_have_been,
    ROUND(ABS(r.billed_at_price - ph.price_per_kwh), 4) AS price_difference,
    r.kwh_value,
    ROUND(r.kwh_value * (r.billed_at_price - ph.price_per_kwh), 2) AS cost_impact
FROM READINGS_15MIN r
-- Get customer who owned meter at time of usage
JOIN CUSTOMER_METER_HISTORY cmh
    ON r.meter_id = cmh.meter_id
    AND r.reading_timestamp BETWEEN cmh.effective_from
        AND COALESCE(cmh.effective_to, '9999-12-31')
-- Get plan customer was on at time of usage
JOIN CUSTOMER_TARIFF_ASSIGNMENT ta
    ON cmh.customer_id = ta.customer_id
    AND r.reading_timestamp BETWEEN ta.effective_from
        AND COALESCE(ta.effective_to, '9999-12-31')
-- Get price that SHOULD have applied at time of usage
JOIN TARIFF_PRICE_HISTORY ph
    ON ta.plan_id = ph.plan_id
    AND r.reading_timestamp BETWEEN ph.effective_from
        AND COALESCE(ph.effective_to, '9999-12-31')
WHERE r.status = 'BILLED'
  AND r.billed_at_price IS NOT NULL
  AND ABS(r.billed_at_price - ph.price_per_kwh) > 0.0001  -- Allow for rounding
  AND r.reading_timestamp >= CURRENT_DATE - INTERVAL 90 DAY
ORDER BY ABS(cost_impact) DESC;

```

Explanation

1. **Temporal joins:** Find customer, plan, and historical price that should have applied at `reading_timestamp`
2. **Price comparison:** `billed_at_price` (what we used) vs `price_per_kwh` (what we should have used)
3. **Cost impact:** Calculate over/under charge amount (`kwh × price_difference`)
4. **Filter:** Only billed readings with price mismatch
5. **Order by impact:** Largest over/under charges first (prioritize corrections)

Sample Output

```
reading_id | meter_id | usage_timestamp     | days_late | price_charged | price_should_have_been | cost_impact
-----------|----------|---------------------|-----------|---------------|------------------------|------------
12345678   | M67890   | 2026-03-15 14:30:00 | 25        | 0.18          | 0.15                   | 15.00
98765432   | M11111   | 2026-03-20 09:00:00 | 18        | 0.15          | 0.18                   | -12.50

```

Business Action

Readings with `cost_impact` should trigger:

- Credit memo (if customer overcharged, negative impact)
- Debit memo (if customer undercharged, positive impact)

Query 3: Show Customers Whose Cost Would Change if Historical Tariff Edited

Reasoning

Scenario: Tariff Manager edits a historical price (e.g., changes price for March 2026 from 0.15 to 0.18)

Impact: All readings already billed in that period will have wrong cost

Solution: Identify affected customers and calculate recalculation amount

SQL Query

```sql
-- Parameters for the hypothetical price change
SET @affected_plan_id = 5;
SET @affected_date_start = '2026-03-01';
SET @affected_date_end = '2026-03-31';
SET @old_price = 0.15;
SET @new_price = 0.18;

SELECT
    c.customer_id,
    c.name AS customer_name,
    c.email,
    COUNT(r.reading_id) AS affected_readings,
    ROUND(SUM(r.kwh_value), 2) AS total_kwh,
    ROUND(SUM(r.kwh_value * @old_price), 2) AS current_cost,
    ROUND(SUM(r.kwh_value * @new_price), 2) AS new_cost,
    ROUND(SUM(r.kwh_value * (@new_price - @old_price)), 2) AS cost_difference
FROM READINGS_15MIN r
JOIN CUSTOMER_METER_HISTORY cmh
    ON r.meter_id = cmh.meter_id
    AND r.reading_timestamp BETWEEN cmh.effective_from
        AND COALESCE(cmh.effective_to, '9999-12-31')
JOIN CUSTOMERS c
    ON cmh.customer_id = c.customer_id
JOIN CUSTOMER_TARIFF_ASSIGNMENT ta
    ON c.customer_id = ta.customer_id
    AND r.reading_timestamp BETWEEN ta.effective_from
        AND COALESCE(ta.effective_to, '9999-12-31')
WHERE r.status = 'BILLED'
  AND ta.plan_id = @affected_plan_id
  AND r.reading_timestamp BETWEEN @affected_date_start AND @affected_date_end
  AND r.billed_at_price = @old_price  -- Only readings billed at old price
GROUP BY c.customer_id, c.name, c.email
HAVING ABS(cost_difference) > 0.01  -- Ignore negligible amounts
ORDER BY ABS(cost_difference) DESC;

```

Explanation

1. **Parameters:** Define which price is being edited (plan, date range, old/new price)
2. **Temporal joins:** Find customers who were on affected plan during affected period
3. **Filter:** Only BILLED readings (not yet recalculated)

4. **Aggregate:** Sum up impact per customer
5. **cost_difference:**
   - Positive = customer owes more (we undercharged)
   - Negative = customer gets refund (we overcharged)
6. **Order by impact:** Largest adjustments first

Sample Output

```
customer_id | customer_name | email              | affected_readings | total_kwh | current_cost | new_cost | cost_difference
------------|---------------|--------------------|-------------------|-----------|--------------|----------|----------------
C12345      | John Smith    | john@email.com     | 2880              | 450.00    | 67.50        | 81.00    | 13.50
C67890      | Jane Doe      | jane@email.com     | 2880              | 320.00    | 48.00        | 57.60    | 9.60
C11111      | Bob Jones     | bob@email.com      | 2880              | 180.00    | 27.00        | 32.40    | 5.40

```

Business Action

1. **Positive cost_difference:** Issue debit memo, charge customer additional amount
2. **Negative cost_difference:** Issue credit memo, refund customer
3. **Approval workflow:** Large amounts (>$100) require Finance approval

Query 4: Standard Deviation of Usage Per Meter

Reasoning

- Standard deviation identifies meters with erratic consumption (potential tampering or faults)
- Calculate variance from mean for each meter
- High standard deviation = inconsistent usage pattern

SQL Query

```sql
SELECT
    meter_id,
    ROUND(AVG(daily_kwh), 2) AS avg_daily_kwh,
    ROUND(STDDEV(daily_kwh), 2) AS stddev_daily_kwh,
    ROUND(STDDEV(daily_kwh) / NULLIF(AVG(daily_kwh), 0) * 100, 1) AS coefficient_of_variation_pct,
    COUNT(DISTINCT usage_date) AS days_counted,
    ROUND(MIN(daily_kwh), 2) AS min_daily_kwh,
    ROUND(MAX(daily_kwh), 2) AS max_daily_kwh
FROM (
    -- Subquery: Aggregate 15-min readings to daily totals
    SELECT
        meter_id,
        DATE(reading_timestamp) AS usage_date,
        SUM(kwh_value) AS daily_kwh
    FROM READINGS_15MIN
    WHERE status IN ('VALID', 'BILLED')
      AND reading_timestamp >= CURRENT_DATE - INTERVAL 90 DAY
```

```
    GROUP BY meter_id, DATE(reading_timestamp)
) daily_totals
GROUP BY meter_id
HAVING days_counted >= 30  -- Need minimum sample size for meaningful stddev
ORDER BY coefficient_of_variation_pct DESC;

```

Explanation

1. **Inner query:** Convert 15-min readings to daily totals (same as Query 1)
2. **STDDEV:** Calculate standard deviation of daily consumption
3. **Coefficient of variation:** `(stddev / mean) × 100` - normalized measure
   - 10-20% = normal variation
   - 50%+ = highly erratic (investigate)
4. **Min/Max:** Show range to understand variance context
5. **Filter:** Require 30+ days of data for statistical validity

Sample Output

```
meter_id | avg_daily_kwh | stddev_daily_kwh | coefficient_of_variation_pct | days_counted | min_daily_kwh | max_daily_kwh
---------|---------------|------------------|------------------------------|--------------|---------------|---------------
M99999   | 45.00         | 35.00            | 77.8                         | 89           | 5.00          | 150.00
M88888   | 120.00        | 25.00            | 20.8                         | 90           | 80.00         | 180.00
M77777   | 30.00         | 3.50             | 11.7                         | 90           | 22.00         | 38.00

```

Interpretation

- **M99999:** 77.8% variation - INVESTIGATE (possible tampering or commercial usage)
- **M88888:** 20.8% variation - Normal (seasonal variation)
- **M77777:** 11.7% variation - Very consistent (residential baseline)

Business Action

- High coefficient (>50%) → Flag for anomaly detection
- Sudden spike in stddev → Meter fault or usage pattern change

Performance Considerations

All queries include:

1. **Partition pruning:** `reading_timestamp >= CURRENT_DATE - INTERVAL 90 DAY`
   - Ensures query only scans recent partitions
   - 90 days = ~4.3B rows instead of full 87.5B (5-year total)
2. **Index usage:**

```sql
-- Required indexes
CREATE INDEX idx_readings_meter_timestamp ON READINGS_15MIN(meter_id, reading_timestamp);
CREATE INDEX idx_readings_status ON READINGS_15MIN(status, reading_timestamp);
CREATE INDEX idx_meter_history_temporal ON CUSTOMER_METER_HISTORY(meter_id, effective_from, effective_to);
CREATE INDEX idx_price_history_temporal ON TARIFF_PRICE_HISTORY(plan_id, effective_from, effective_to);

```

3. **Temporal join optimization:**
   - `BETWEEN effective_from AND COALESCE(effective_to, '9999-12-31')` uses index range scan
   - Much faster than `effective_from <= date AND (effective_to IS NULL OR effective_to >= date)`
4. **Aggregation strategy:**
   - Daily aggregation in subquery reduces dataset before outer calculations
   - 48M rows/day → ~4.3M daily totals (90 days) = 99% reduction

Key Design Decisions

Decision	Reasoning
90-day window	Balance between data completeness and query performance
billed_at_price field	Enables "wrong price" detection without recalculating from history
Coefficient of variation	Normalized stddev measure - comparable across different usage levels
Temporal joins	Required to answer "what should price have been" vs "what price was used"
HAVING days_counted >= 30	Statistical validity - stddev meaningless with small samples
Filter on status	Only VALID/BILLED readings - excludes errors and rejections from calculations